Contents

Heap / Priority Queue — Complete Interview Handbook

Pattern #17 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Heap section

---

Table of Contents

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

A Heap is a complete binary tree (usually array-backed) satisfying the **heap property**: every parent is <= (min-heap) or >= (max-heap) all its children. This gives O(1) access to the minimum/maximum element and O(log n) insertion/removal, making it the go-to structure for **"repeatedly get the smallest/largest"** problems — Top-K, K-way merge, running median, and scheduling by priority.

1.2 Why Was This Pattern Invented?

Fully sorting data just to repeatedly extract the current minimum/maximum is wasteful — O(n log n) upfront when you might only need the top few elements, or need to interleave insertions with extractions dynamically (as in real-time scheduling). A Heap was invented to support **O(log n) insert AND O(log n) extract-min/max simultaneously**, without maintaining full sorted order — a weaker, cheaper invariant (heap property) that's just strong enough to always know the current extreme in O(1).

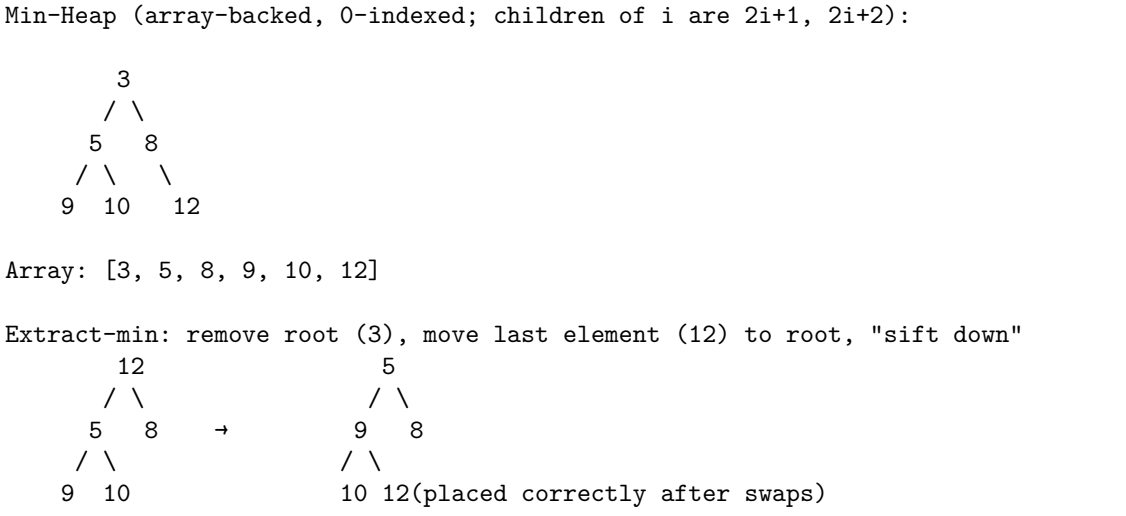
1.3 Real Intuition Behind The Pattern

Think of a **hospital emergency room triage system** — patients aren't served first-come-first-served; the most critical patient is always treated next, and new arrivals are inserted into the right priority position without needing to re-sort the entire waiting room every time someone new arrives or is treated.

1.4 Mental Model

The heap only guarantees "parent is more extreme than its children" — NOT full sorted order between siblings or across levels. This weaker guarantee is exactly what makes both insertion and removal cheap (O(log n), just fixing one root-to-leaf path), while still always exposing the single most extreme element at the root in O(1).

1.5 Visual Explanation



1.6 Simple Analogy

A Heap is like a **single-elimination tournament bracket viewed upside down** — the "winner" (min or max) is always at the top, easily accessible, but the relative order among all the "losers" further down the bracket is only loosely constrained (each only needs to have lost to someone "better" directly above them, not to everyone).

1.7 When Should I Immediately Think About Using This Pattern?

- "Top K largest/smallest," "Kth largest/smallest element."
- "Merge K sorted lists/arrays."
- "Running median" from a data stream.
- "Task scheduling by priority" (CPU scheduling, meeting rooms via end-time heap).
- Any problem needing **repeated access to the current min/max** while the data set is dynamically changing (insertions/removals interleaved with extreme-value queries).

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"kth largest/smallest"	Direct signal
"top k"	Direct signal
"merge k sorted lists"	K-way merge via heap
"running median"	Two-heap technique
"closest points to origin"	Max-heap of size k
"task scheduler," "meeting rooms"	Priority-based scheduling via heap

2.2 Hidden Hints

"Top K" or "Kth" phrasing where K is much smaller than N is the single strongest tell — full sorting (O(n log n)) is overkill when a heap of size K can solve it in O(n log K).

2.3 Interview Clues

Interviewer asks "can you avoid sorting the entire array?" after you propose a full-sort solution for a top-K style problem.

2.4 Common Trick Words

"Kth," "top," "closest," "smallest range covering," "reorganize by frequency" (bucket sort is an alternative here, but heaps work too) — all point to repeated extreme-value access.

2.5 What Interviewers Expect

Correct choice of min-heap vs max-heap (often counterintuitive — e.g., a min-heap of size K for "top K largest," since you want to efficiently discard the smallest of your current top-K candidates), and the O(n log K) vs O(n log n) complexity trade-off articulated explicitly.

2.6 When NOT To Use This Pattern

- You need **full sorted order**, not just the top few or repeated extremes — just sort directly (O(n log n)), a heap-based approach doesn't help here.

- The value range is **small and bounded** — Counting Sort/Bucket Sort can achieve O(n) instead of a heap's O(n log k).
- You need **arbitrary order statistics** (not just min/max) repeatedly — a balanced BST or order-statistics tree may be more appropriate than a heap, which only efficiently exposes the single extreme.

SECTION 3 — Decision Framework

```
Do you need REPEATED access to the current min/max while data changes dynamically?

       Yes

Do you need the TOP K (K << N) rather than a full sort?

       Yes → USE A HEAP of size K (O(n log K), better than full O(n log n) sort)

        No

Do you need to MERGE K SORTED sequences?

       Yes → USE A MIN-HEAP for K-way merge (O(n log k), n = total elements, k = number of sequences)

        No

Do you need a RUNNING MEDIAN or a value at a specific percentile, dynamically updated?

       Yes → USE TWO HEAPS (max-heap for lower half, min-heap for upper half)

        No

Is the value range SMALL and BOUNDED?

       Yes → Consider COUNTING SORT / BUCKET SORT instead (O(n), better than a heap's O(n log k))
```

Why: A heap's specific value is maintaining efficient access to a *single* running extreme (or a small top-K set) under dynamic insertion/removal — for a full sort, a one-time O(n log n) sort is simpler and equally fast; for bounded small ranges, counting-based approaches beat a heap's O(log k) per-operation overhead entirely.

SECTION 4 — Why This Pattern Works

Mathematical/Logical: A binary heap of `n` elements has height `O(log n)` because it's a **complete** binary tree (every level fully filled except possibly the last, filled left to right) — this structural guarantee (not a balancing algorithm, just the definition of "complete") bounds insertion/removal to `O(log n)`: both operations only ever need to fix a single root-to-leaf (or leaf-to-root) path, of length `O(log n)`.

Intuitive: Since the heap only enforces "parent more extreme than children" (not full ordering across the whole tree), fixing the structure after an insertion or removal only requires "bubbling" the affected element up or down one path — a much weaker (and cheaper) invariant to maintain than full sortedness.

Correctness Proof (heap property maintains min/max at root): *Invariant:* for every node `v` in the heap, `v.value <= child.value` for all its children (min-heap case). *Base case:* a single-node

heap trivially satisfies this. *Inductive step:* insertion places the new element at the next available leaf position, then repeatedly swaps it with its parent while it violates the heap property, terminating when the invariant is restored at every level along that path — by induction, this restores the invariant globally, since only the single path from the new leaf to the root was potentially violated. *Consequence:* since every node satisfies `parent <= children` transitively down every path, the root — having no parent to be <= — must be <= every other node in the heap, i.e., the global minimum. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Top K Largest via Min-Heap of Size K)

1. Initialize an empty min-heap.
2. For each element: push it onto the heap.
3. If the heap size exceeds K, pop the minimum (discard the smallest of the current top-K candidates).
4. After processing all elements, the heap contains exactly the K largest elements (the min-heap's root being the Kth largest).

5.2 General Template — Top K via Min-Heap

```
function topKLargest(arr, k):
    minHeap = new MinHeap()
    for num in arr:
        minHeap.push(num)
        if minHeap.size() > k:
            minHeap.pop()                  # discard current smallest among top-K candidates
    return minHeap.toArray()               # contains the K largest elements
```

5.3 K-Way Merge Template

```
function mergeKSortedLists(lists):
    minHeap = new MinHeap()                 # stores (value, listIndex, elementIndex)
    for i, list in enumerate(lists):
        if list is not empty:
            minHeap.push((list[0], i, 0))

    result = []
    while minHeap is not empty:
        value, listIdx, elemIdx = minHeap.pop()
        result.append(value)
        if elemIdx + 1 < length(lists[listIdx]):
            minHeap.push((lists[listIdx][elemIdx+1], listIdx, elemIdx+1))

    return result
```

5.4 Running Median Template (Two Heaps)

```
function addNum(num, maxHeapLower, minHeapUpper):
    if maxHeapLower is empty or num <= maxHeapLower.peek():
        maxHeapLower.push(num)
    else:
        minHeapUpper.push(num)
```

```
    # rebalance so sizes differ by at most 1
    if maxHeapLower.size() > minHeapUpper.size() + 1:
        minHeapUpper.push(maxHeapLower.pop())
    else if minHeapUpper.size() > maxHeapLower.size():
        maxHeapLower.push(minHeapUpper.pop())

function findMedian(maxHeapLower, minHeapUpper):
    if maxHeapLower.size() > minHeapUpper.size():
        return maxHeapLower.peek()
    return (maxHeapLower.peek() + minHeapUpper.peek()) / 2.0
```

5.5 Interview Thinking Process

1. "This needs repeated access to the current min/max under dynamic changes — I'll use a heap rather than a full sort."
2. "For 'top K largest,' I'll counterintuitively use a MIN-heap of size K — it lets me efficiently discard the smallest of my current top-K candidates whenever a bigger one arrives."
3. "For merging K sorted sequences, I'll use a min-heap holding one 'current candidate' per sequence, always extracting the global minimum next."
4. "For a running median, I'll split the data into two heaps (max-heap for the lower half, min-heap for the upper half), keeping them balanced in size."
5. "I'll state the complexity as O(n log k), better than a full O(n log n) sort when k « n."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	O(log n) per insert/extract; O(n log n) to heapify n elements one at a time; O(n) to build a heap from an existing array (heapify)	O(n) for the heap itself	Each insert/extract fixes at most one root-to-leaf path, O(log n) long
Average Case	Same as worst case — heap operations don't depend on data distribution	O(n)	Deterministic structural guarantee (complete binary tree)
Best Case	O(1) to peek the min/max	O(n)	Peeking the root is always O(1), regardless of heap size
Amortized	O(log n) per operation across a sequence of n operations	O(n)	No special amortization beyond the standard O(log n) bound (unlike some advanced heap variants like Fibonacci heaps)

Build-heap subtlety: building a heap from `n` elements all at once (heapify) is O(n), NOT O(n log n) — a classic, non-obvious result from the fact that most nodes are near the bottom of the tree

and require very little sifting.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty heap, peek/pop	No elements	Must guard against underflow — return null/error/exception per language convention
Single element	`[5]`	Trivially the min and max simultaneously
K larger than array size (Top-K problems)	`k=10`, array has 5 elements	Return all elements, or clarify problem constraints explicitly
Duplicate values	`[3,3,3]`	Heap correctly handles duplicates; comparator must define consistent tie-breaking if needed
All elements identical	`[5,5,5,5]`	Heap property trivially holds regardless of structure
Merging K lists where some are empty	`[[], [1,2], []]`	Must skip empty lists when initializing the heap, avoid pushing from an empty list
Running median with heaps becoming unbalanced	Uneven insertion pattern	Rebalancing step must run after every insertion, not just occasionally

Common mistakes: using a max-heap when a min-heap of size K was needed for "top K largest" (leads to O(n log n) instead of O(n log K) — still correct, but suboptimal, and reveals a misunderstanding of the technique); forgetting to rebalance the two-heap running-median structure after every single insertion, causing skewed medians.

SECTION 8 — Pros & Cons

Advantages: O(log n) insert/extract for dynamic priority-based access; O(1) peek at the current extreme; O(n) heapify from an existing array; well-suited for streaming/dynamic data where full re-sorting on every change would be wasteful. **Disadvantages:** No efficient support for arbitrary order-statistics queries (only the single extreme is O(1)); no efficient arbitrary-element deletion (only extreme removal) without additional bookkeeping (lazy deletion or indexed heaps). **Trade-offs:** Heap (O(log n) per operation, dynamic) vs. Full Sort (O(n log n) once, static thereafter) vs. Balanced BST (O(log n) per operation, supports arbitrary order-statistics but higher constant factor) — choose based on whether you need just the extreme (heap) or arbitrary rank queries (BST). **Limitations:** Not a substitute for a fully sorted structure when arbitrary range/rank queries are needed. **Inefficient when:** the value range is small and bounded (counting/bucket sort beats a heap's O(log k) per-operation overhead with O(1) amortized instead).

SECTION 9 — Real World Applications

Domain	Application
Operating Systems	CPU process scheduling by priority (priority queues are literally heaps in many scheduler implementations)
Google/Amazon/Meta	Top-K trending topics/search results/recommendations computed via heap-based Top-K selection
Networking	Packet scheduling by priority/QoS class using priority queues
Dijkstra's/Prim's Algorithms	Both rely fundamentally on a min-heap for efficient "next closest/cheapest" selection (Pattern #21, #19-adjacent)
Event-Driven Simulation	Discrete event simulators use a min-heap of upcoming events ordered by timestamp
Huffman Coding (Compression)	Building an optimal prefix-free code uses a min-heap to repeatedly merge the two least-frequent nodes
Load Balancers	Selecting the least-loaded server (min-heap by current load) for request routing
Financial Trading Systems	Order book "best price" queries use heap-like priority structures for buy/sell order matching
Real-Time Systems	Task scheduling with deadlines uses priority queues to always execute the most urgent task next
Streaming Analytics	Running median/percentile computations over live data streams via the two-heap technique

---

SECTION 10 — Interview Strategy

How seniors answer: They correctly (and counterintuitively) choose a min-heap of size K for "top K largest" problems, explicitly state the O(n log K) vs O(n log n) trade-off, and proactively mention alternatives (counting sort for bounded ranges, quickselect for a one-time O(n) average-case selection without needing a full heap).

How juniors answer: They often default to sorting the entire array even when only the top K elements are needed, or they use a max-heap of the full size N (defeating the space/time benefit of a bounded-size heap).

Typical follow-ups: "Can you avoid a full sort?" "What if K is very close to N — does the heap approach still make sense?" (Discuss when the heap's advantage shrinks as K approaches N.) "Can you find the Kth largest without a heap, using expected O(n) time?" (Quickselect — a Two-Pointers/partition-based alternative).

Optimization questions: "Can you build the heap in O(n) instead of O(n log n)?" (Yes — heapify an existing array directly, rather than inserting one element at a time.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Top-K (Min-Heap of size K)	Efficiently track K largest elements	Kth Largest Element in an Array, Top K Frequent Elements
Top-K (Max-Heap of size K)	Efficiently track K smallest elements	K Closest Points to Origin
K-Way Merge	Merge multiple sorted sequences	Merge K Sorted Lists
Two Heaps (Running Median)	Split into lower/upper halves for dynamic median	Find Median from Data Stream
Heap + Greedy (Scheduling)	Priority-based task/interval scheduling	Task Scheduler, Meeting Rooms II
Heap for Dijkstra's/Prim's	Priority-based graph algorithms	Network Delay Time, Minimum Spanning Tree
Indexed/Lazy-Deletion Heap	Supports efficient arbitrary-element removal via marking	Sliding Window Median (heap-based alternative)

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Sorting	Full order, O(n log n), static	Need complete sorted order, not just extremes
Quickselect	O(n) average-case for a single Kth-order-statistic query, no ongoing dynamic structure	One-off Kth element query, not repeated/dynamic access
Balanced BST	O(log n) per operation, supports arbitrary rank/range queries	Need more than just the single extreme (e.g., arbitrary percentile, range count)
Monotonic Deque	O(n) total for sliding-window min/max, but only within a fixed window, not general dynamic priority	Sliding window extremes specifically, not general priority queue needs

Comparison Table

Aspect	Heap	Full Sort	Quickselect
Time (Top-K)	O(n log k)	O(n log n)	O(n) average
Supports dynamic insertion	Yes	No (static)	No (one-off query)
Space	O(k) or O(n)	O(1) to O(n)	O(1)

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct single-heap usage	Kth Largest Element in a Stream, Last Stone Weight
Medium	Top-K with custom comparators, K-way merge	Kth Largest Element in an Array, Top K Frequent Elements, K Closest Points to Origin
Hard	Two-heap techniques, complex scheduling	Find Median from Data Stream, Merge K Sorted Lists, Task Scheduler
Very Hard	Multi-constraint heap combinations, advanced graph algorithms	IPO, Smallest Range Covering Elements from K Lists, Network Delay Time (Dijkstra's)

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Kth Largest Element in an Array	Medium	Amazon, Meta, Microsoft, Google	Min-heap of size K	Foundational Top-K mechanics
2	Kth Largest Element in a Stream	Easy	Amazon, Meta	Persistent min-heap of size K across insertions	Streaming Top-K
3	Top K Frequent Elements	Medium	Amazon, Meta, Google	Frequency map + heap (or bucket sort alternative)	Frequency + heap combination
4	K Closest Points to Origin	Medium	Amazon, Meta, Google	Max-heap of size K by distance	Distance-based Top-K
5	Merge K Sorted Lists	Hard	Amazon, Meta, Google, Microsoft	Classic K-way merge via min-heap	K-way merge mastery
6	Find Median from Data Stream	Hard	Amazon, Meta, Google, Microsoft	Two-heap technique	Advanced dual-heap mastery
7	Task Scheduler	Medium	Amazon, Meta, Google	Max-heap-based greedy scheduling	Heap + greedy combination
8	Meeting Rooms II	Medium	Amazon, Meta, Google, Microsoft	Min-heap of end times for concurrent counting	Cross-pattern (Intervals + Heap)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
9	Last Stone Weight	Easy	Amazon	Direct max-heap simulation	Basic max-heap simulation
10	Reorganize String	Medium	Amazon, Google	Max-heap-based greedy character placement	Heap + greedy string construction
11	Sort Characters By Frequency	Medium	Amazon, Google	Frequency map + heap-based sorting	Frequency + heap combination
12	Ugly Number II	Medium	Amazon, Google	Min-heap for ordered generation (or DP alternative)	Heap-based sequence generation
13	Super Ugly Number	Medium	Google	Min-heap generalization of Ugly Number II	Advanced heap-based generation
14	Smallest Range Covering Elements from K Lists	Hard	Google, Amazon	Multi-pointer + min-heap combination	Advanced multi-list heap combination
15	IPO	Hard	Amazon, Google	Two-heap (or sort + heap) greedy optimization	Advanced heap + greedy combination
16	Sliding Window Median	Hard	Amazon, Google	Two heaps with lazy deletion for window sliding	Advanced heap + window combination
17	Network Delay Time	Medium	Amazon, Google, Meta	Dijkstra's algorithm using a min-heap	Cross-pattern (Heap + Graph Shortest Path)
18	Cheapest Flights Within K Stops	Medium	Amazon, Google	Modified Dijkstra's/Bellman-Ford with heap	Cross-pattern (Heap + Graph)
19	Path With Minimum Effort	Medium	Google, Amazon	Dijkstra's-style min-heap graph traversal	Cross-pattern (Heap + Graph)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
20	Swim in Rising Water	Hard	Google, Amazon	Min-heap-based Dijkstra's-style grid traversal	Cross-pattern (Heap + Grid Graph)
21	Kth Smallest Element in a Sorted Matrix	Medium	Amazon, Google	Heap-based or binary-search-based selection	Alternative technique comparison
22	Find K Pairs with Smallest Sums	Medium	Amazon, Google	Min-heap for combinatorial pair generation	Advanced heap-based generation
23	Trapping Rain Water II	Hard	Google, Amazon	Min-heap-based boundary expansion (2D)	Advanced 2D heap application
24	The Skyline Problem	Hard	Google, Amazon, Meta	Max-heap (with lazy deletion) for building height tracking	Advanced sweep line + heap
25	Design Twitter	Medium	Amazon, Meta, Google	Heap-based feed merging	System-design-adjacent heap application
26	Maximum Performance of a Team	Hard	Google, Amazon	Sort + min-heap for constrained optimization	Advanced sort + heap combination
27	Minimum Cost to Hire K Workers	Hard	Google, Amazon	Sort + max-heap for ratio-based optimization	Advanced sort + heap combination
28	Single-Threaded CPU	Medium	Amazon, Google	Min-heap-based task simulation	Simulation + heap combination
29	Process Tasks Using Servers	Medium	Amazon, Google	Dual min-heap (free/busy servers) simulation	Advanced dual-heap simulation
30	Maximum Average Pass Ratio	Medium	Google, Amazon	Greedy + max-heap for marginal-gain optimization	Advanced greedy + heap combination

SECTION 15 — Common Mistakes

1. Using a max-heap of the full size N for "top K largest" instead of a min-heap of size K, losing the space/time advantage. *Fix:* always use the "opposite" heap type bounded to size K — min-heap for largest-K, max-heap for smallest-K.
2. Forgetting that building a heap from an existing array (heapify) is O(n), not O(n log n) — understating your own solution's efficiency. *Fix:* use the language's built-in heapify function when available, and state the correct O(n) build complexity.
3. Not rebalancing the two-heap running-median structure after every insertion, causing the heaps to drift out of the required size balance. *Fix:* always rebalance immediately after each insertion, not periodically.
4. Attempting arbitrary-element deletion from a heap without additional bookkeeping (a heap only efficiently supports extreme-element removal) — a common oversight in "sliding window" heap-based problems. *Fix:* use lazy deletion (mark as invalid, skip when popped) or an indexed heap supporting O(log n) arbitrary removal.
5. Defaulting to a heap-based solution when the value range is small/bounded, missing a faster O(n) counting/bucket sort alternative. *Fix:* always check whether the value range permits a non-comparison-based approach first.

Why people fail: the "min-heap for largest-K" choice is genuinely counterintuitive on first exposure, and candidates who haven't internalized *why* (you want to efficiently discard the smallest of your current top-K set) often either get the heap type backwards or fall back to full sorting, missing the intended optimization.

SECTION 16 — Optimization Techniques

- **Time:** Use O(n) heapify to build a heap from an existing array rather than inserting elements one at a time (O(n log n)); use Quickselect instead of a heap for a one-off (non-streaming) Kth-element query, achieving O(n) average time.
- **Space:** Bound the heap to size K explicitly for Top-K problems rather than holding all N elements.
- **Readability:** Clearly comment the counterintuitive heap-type choice (e.g., "min-heap here because we want to efficiently discard the smallest of our top-K candidates").
- **Interview performance:** Proactively state the O(n log k) vs O(n log n) trade-off and mention Quickselect/counting-sort alternatives — demonstrating breadth of technique selection, not just heap mechanics.

SECTION 17 — Multiple Language Templates

Java

```java
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) minHeap.poll();
    }
    return minHeap.peek();
}
```

JavaScript

```javascript
// Using a simple array-based min-heap simulation via sorted insertion for illustration;
// production code should use a proper heap implementation or a library.
function findKthLargest(nums, k) {
    const minHeap = [];
    for (const num of nums) {
        minHeap.push(num);
        minHeap.sort((a, b) => a - b);   // illustrative only; use a real heap for O(log n)
        if (minHeap.length > k) minHeap.shift();
    }
    return minHeap[0];
}
```

PHP

```php
function findKthLargest(array $nums, int $k): int {
    $minHeap = new SplMinHeap();
    foreach ($nums as $num) {
        $minHeap->insert($num);
        if ($minHeap->count() > $k) $minHeap->extract();
    }
    return $minHeap->top();
}
```

Python

```python
import heapq
def find_kth_largest(nums, k):
    min_heap = []
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return min_heap[0]
```

Go

```go
import "container/heap"

type MinHeap []int
func (h MinHeap) Len() int            { return len(h) }
func (h MinHeap) Less(i, j int) bool  { return h[i] < h[j] }
func (h MinHeap) Swap(i, j int)       { h[i], h[j] = h[j], h[i] }
func (h *MinHeap) Push(x interface{}) { *h = append(*h, x.(int)) }
func (h *MinHeap) Pop() interface{} {
    old := *h
    n := len(old)
    x := old[n-1]
    *h = old[:n-1]
    return x
}
func findKthLargest(nums []int, k int) int {
```

```
    h := &MinHeap{}
    heap.Init(h)
    for _, num := range nums {
        heap.Push(h, num)
        if h.Len() > k {
            heap.Pop(h)
        }
    }
    return (*h)[0]
}
```

C++

```
int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int num : nums) {
        minHeap.push(num);
        if ((int)minHeap.size() > k) minHeap.pop();
    }
    return minHeap.top();
}
```

SECTION 18 — Dry Runs

Small Input

`nums = [3, 2, 1, 5, 6, 4]`, `k = 2` (2nd largest)

```
push 3: heap=[3]
push 2: heap=[2,3]
push 1: heap=[1,2,3] → size(3)>k(2) → pop 1 → heap=[2,3]
push 5: heap=[2,3,5] → pop 2 → heap=[3,5]
push 6: heap=[3,5,6] → pop 3 → heap=[5,6]
push 4: heap=[4,5,6] → pop 4 → heap=[5,6]
Final heap root (min) = 5 → 2nd largest = 5   (sorted: [1,2,3,4,5,6], 2nd largest is indeed 5)
```

Large Input (Conceptual)

For an array of 10^6 elements with K=10, the heap approach costs O(10^6 × log(10)) ≈ 3.3×10^6 operations, versus a full sort's O(10^6 × log(10^6)) ≈ 2×10^7 — roughly 6x fewer operations, illustrating the practical benefit of bounding the heap to size K.

Corner Case

`nums = [5]`, `k = 1`: `push 5` → heap=[5], size(1) not > k(1) → no pop → root = 5, correctly the "1st largest" (and only) element.

SECTION 19 — Advanced Concepts

- **O(n) Heapify:** building a heap from an unordered array by starting from the last non-leaf node and sifting down each node in reverse level order achieves O(n) total, not O(n log n) —

a classic, non-obvious amortized analysis result (most nodes are near the bottom and require minimal sifting).

- **Indexed/Lazy-Deletion Heaps:** for problems needing arbitrary-element removal (e.g., Sliding Window Median), maintain a hashmap tracking "invalidated" elements, and skip/discard them lazily when they reach the top of the heap during a pop — avoiding the need for a full indexed heap implementation in most interview contexts.
- **Heap + Greedy hybrid (Task Scheduler, IPO):** many "hard" heap problems are really greedy algorithms where a heap is simply the efficient data structure enabling the greedy choice (e.g., "always pick the most frequent remaining task" or "always pick the most profitable affordable project") to be made in O(log n) instead of O(n) per step.
- **Fibonacci Heaps (theoretical):** offer O(1) amortized insert and decrease-key operations (versus a binary heap's O(log n)), which theoretically improves Dijkstra's algorithm's complexity — rarely implemented in interviews due to complexity, but worth mentioning as an advanced awareness point for Staff-level discussions.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize the heap as the data structure of choice whenever a problem needs **"always process the most extreme/urgent item next," with the item set changing dynamically** — the same abstraction underlies OS process scheduling, event-driven simulation, and real-time task queues in distributed systems. They're also quick to recognize when a heap is *not* the optimal choice — bounded small value ranges (counting sort), one-off queries (Quickselect), or when full sorted order is genuinely needed (just sort). Interviewers evaluate whether a candidate can correctly identify the "min-heap for top-K-largest" counterintuitive choice and explain *why*, and whether they know the O(n) heapify result rather than assuming heap construction is always O(n log n).

SECTION 21 — Cheat Sheet

```
PATTERN: Heap / Priority Queue
RECOGNIZE: "top K," "kth largest/smallest," "merge K sorted," "running median," priority-based scheduling
TEMPLATE (Top-K largest, min-heap of size K):
    minHeap = []
    for num in arr:
        push(minHeap, num)
        if size(minHeap) > k: pop(minHeap)
    return minHeap  # contains K largest; root is the Kth largest
COMPLEXITY: O(log n) per insert/extract; O(n) to heapify; O(1) to peek
KEY PROOF: complete binary tree structure bounds height to O(log n), so fixing one root-to-leaf path is O
WATCH FOR: min-heap vs max-heap choice (counterintuitive for top-K-largest), O(n) heapify (not O(n log n))
DOESN'T APPLY WHEN: need full sorted order (just sort), bounded small value range (use counting/bucket sor
```

SECTION 22 — Revision Notes (5-Minute Review)

- Heap property: parent more extreme than children — weaker than full sort, cheaper to maintain (O(log n) vs O(n log n)).
- Top-K largest → min-heap of size K (counterintuitive but correct — discard the smallest of your candidates).
- K-way merge → min-heap holding one candidate per sequence.
- Running median → two heaps (max-heap lower half, min-heap upper half), rebalance after every insertion.

- Heapify an existing array is O(n), not O(n log n).
- Heaps don't support efficient arbitrary-element deletion without extra bookkeeping (lazy deletion/indexed heap).
- Bounded small value ranges: prefer counting/bucket sort over a heap.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic heap usage	Kth Largest Element in a Stream (703), Last Stone Weight (1046)
Intermediate	Top-K variants, custom comparators	Kth Largest Element in an Array (215), Top K Frequent Elements (347), K Closest Points to Origin (973)
Advanced	K-way merge, two-heap median	Merge K Sorted Lists (23), Find Median from Data Stream (295), Task Scheduler (621)
Expert	Cross-pattern (Heap + Graph), advanced scheduling	Network Delay Time (743), IPO (502), Sliding Window Median (480)

SECTION 24 — Memory Tricks

- **Mnemonic:** "**O**pposite heap for Top-K" (min-heap for largest, max-heap for smallest) — remember it's always the *opposite* extreme type.
- **Visualization:** A **hospital triage system** — the most critical patient is always accessible at the top, without re-sorting the whole waiting room.
- **Recognition shortcut:** "Top K / Kth / merge K sorted / running median" → Heap, and remember the counterintuitive min-heap-for-largest-K rule.

SECTION 25 — Final Summary

A Heap maintains the weaker (and cheaper) "parent more extreme than children" invariant instead of full sorted order, giving O(1) access to the current extreme and O(log n) insertion/removal — exactly the right trade-off for dynamic Top-K, K-way merge, and running-median problems. The single most important thing to remember forever: **for "top K largest" problems, use a min-heap bounded to size K (not a max-heap of the full size), since you want to efficiently discard the smallest of your current candidates whenever a bigger one arrives** — and remember that heapify from an existing array is O(n), not O(n log n), a detail that often surprises candidates who assume all heap construction costs the same as repeated insertion.